

Schematron Query Language Binding and XSLT

October 17, 2022

Erik Siegel (erik-siegel@erik-siegel.com)

Schematron Query Language Binding is a lightweight, elegant, and easy-to-use tool for validating XML documents.

Table of Contents

- [1 Introduction](#)
- [2 Query language binding in general](#)
- [3 Using XSL in Schematron](#)
- [4 Using XSLT in Schematron](#)
- [5 Wrap up](#)

1 Introduction

Schematron is one of the XML validation languages and is a lightweight, elegant, and easy-to-use tool for validating XML documents and their metadata. Schematron can be found at <http://www.xml.com/pub/a/2000/11/12/schematron/>.

Any Schematron schema consists of one or more rules, each of which is a logical expression that evaluates to a boolean value (true or false). It should be used as the basis for building a Schematron schema. Schematron is actually a collection of rules that are used to validate XML documents. It is a lightweight, elegant, and easy-to-use tool for validating XML documents.

This concept is a language for describing the structure of XML documents. It is a lightweight, elegant, and easy-to-use tool for validating XML documents. It is a collection of rules that are used to validate XML documents.

This is a lightweight, elegant, and easy-to-use tool for validating XML documents. It is a collection of rules that are used to validate XML documents.

Query Language Binding is a lightweight, elegant, and easy-to-use tool for validating XML documents. It is a collection of rules that are used to validate XML documents.

<http://www.xml.com/pub/a/2000/11/12/schematron/>

2 Query language binding in general

The Query Language Binding is a lightweight, elegant, and easy-to-use tool for validating XML documents. It is a collection of rules that are used to validate XML documents.

Figure 1: A Schematron schema using the `ptd:qahmchmf` attribute to set the Query Language Binding

```
schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3"
```

/schema


```

1    xml    ersion="1. " encoding=" T - "
2    schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3"
3    pattern
      rule context="ordered-item"
        let name="item-id" value=" id-ref"
        assert test="exists(/*/item id eq $item-id )"
          The referenced item value-of-select="$item-id"/ does not exist
        /assert
9    /rule
1   /pattern
11  /schema

```

This element is the identifier key in the schema. This is used to check a schema element with the same identifier in the schema.

But what if the schema contains thousands of elements? A schema with thousands of elements causes a schema error, so a schema with thousands of elements is not efficient and is a pain to maintain.

A solution to this problem is a data model that allows fast lookup of elements. XSLT has an instruction for this: `key()`. It is a query language. You can use this to find elements in a schema.

Figure 4: Schematron schema that checks the identifier references for Figure 2 with a key.

```

1    xml    ersion="1. " encoding=" T - "
2    < -- 1 -   efine the XSLT namespace: -->
3    schema xmlns="http://purl.oclc.org/dsdl/schematron"
      xmlns:xsl="http://www.w3.org/1999/XSL/Transform" queryBinding="xslt3"

      < -- 2 -   efine a key using the XSLT key instruction: -->
        xsl:key name="item-ids" match="*/item" use=" id"

9    pattern
1   rule context="ordered-item"
11  < -- 3 -   eference the key using the key() function: -->
12  assert test="exists(key('item-ids' id-ref))"
13  The referenced item value-of-select=" id-ref"/ does not exist
1   /assert
1   /rule
1   /pattern
1   /schema

```

1 To be a valid schema, the XSLT namespace must be declared. Hence the schema must have the namespace declaration: `xmlns:xsl="http://www.w3.org/1999/XSL/Transform"`.

2 The XSLT `key()` instruction has the following components:

- The name of the key, in this case `item-ids`.
- The nodeset to search, in this case `*/item`.


```

:      xml      ersion="1. "  encoding=" T - "
: < -- 1 -   efine the XSLT namespace on the root element: -->
:      schema xmlns="http://purl.oclc.org/dsdl/schematron"
:      xmlns:xsl="http://www.w3.org/1999/XSL/Transform" queryBinding="xslt3"

< -- 2 -   efine a namespace for the functions as an <ns> element: -->
      nsuri="#functions" prefix="f"/

! < -- 3 -   efine your function using XSLT: -->
1      xsl:function name="f:get-price" as="xs:dou le"
11      xsl:param name="type" as="xs:string"/
12      xsl: aria lname="prices-document" as="document-node()"
13      select="doc('type-codes-and-prices.xml')"/
1      xsl: aria lname="data-element-for-type" as="element(data) "
1      select="$prices-document//data type eq $type/ "
1      xsl:choose
1      xsl:when test="exists($data-element-for-type)"
1      xsl:sequence select="xs:dou le($data-element-for-type/ price)"
19      /xsl:when
2      xsl:otherwise
21      xsl:sequence
22      select="xs:dou le($prices-document/type-codes-and-prices/ default-price)"
23      /xsl:otherwise
2      /xsl:choose
2      /xsl:function
2
2      pattern
2      rule context="thing"
29      < -- 4 - Use the defined function to get the price: -->
3      let name="expected-price" alue="f:get-price( type)"/
31      assert test="$expected-price eq xs:dou le( price)"
32      The price falue-ofselect=" name"/ should e
33      alue-ofselect="$expected-price"/
3      /assert
3      /rule
3      /pattern
3
3      /schema

```

1 We'er g o d n g s t x s d l e t a s o f p , a r e m b e d d e d i n t h e f o s y h e n a d e f i n e T t r h æ m x s p l
on to het re l w k n r k (g s s o . . v v v - v 2 - n q f . 0 . W R . S q m r e n q l !) .

2 X P a t h f u n c t i o n s n a m e s m u s t b e o n o y o n e t d a r n e s p a c e h a n m s r o t
e l e m e n t . T h i s a d u l o o t u s s e y t h i s n a m e a s p a c e s p i o n n s t h i e n X t P h e e s x c a l m e p l æ . n a i
(e t m b s h n m r) a r e d f x p r (u s e e d a r e d o m e x a m p l e a n y s e o u a l y i t k i n g y

3 D e f n o e u r y f u n c t i o n (T s) p o g a m i n g g t h a n g s a g e . I n t h i s e x f a s o p h i d e . t

4 W e u s e t h e f o s e h o l d e d f u n c o t i g o e n t t t h e e x o n e f i t e u d a p d i u s e f r h t s s t i e
e x p e s s i o n .

5W rap up

- Query Language Binding, in which each attribute has a unique language schema.
- In most cases, `works1` and `works2` bindings support
- You specify Language Binding schema as a property of the element of the schema.
- If you don't specify this attribute, the default is
- The `works` binding is not limiting.
- The `works1` and `works2` bindings support complex schemas.

Article contents © 2022 Erik Siegel

© Textuality Services, Inc. except for trademarks appearing on the page.